ПРАКТИЧЕСКАЯ РАБОТА «ОБЪЕКТЫ И МАССИВЫ. МЕТОДЫ ПЕРЕБОРА МАССИВОВ (MAP, FILTER, FOREACH). JSON»

***Цель**: научиться эффективно управлять данными с помощью методов перебора массивов (map, filter, forEach), освоить принципы работы с объектами и форматом JSON, а также реализовать сохранение состояния приложения в localStorage для решения типичных задач Junior-разработчика.*

Подготовка проекта (как в реальной работе)

1. Создайте папку проекта: `js-practice-04`
2. Откройте папку в VS Code.
3. Чтобы не тратить время на CSS, используйте готовую верстку из файла `index.html` и оформление из файла `style.css`. Основная работа выполняется в файле: `script.js`.
4. Создайте репозиторий GitHub с названием `js-practice-04` и загрузите туда проект.

Структура репозитория

```
js-practice-04/
├── index.html
├── style.css
├── script.js
├── screenshots/
│   └── (ваши скриншоты)
└── README.md
```

В **README.md** кратко опишите:

- Название проекта
- Используемые технологии (HTML, CSS, JS)
- Ответы на контрольные вопросы (смотри соответствующий раздел)

Общие требования

- Эта работа проверяет не только наличие работающего кода, а также и ваше ЛИЧНОЕ ПОНИМАНИЕ.
- Код должен быть аккуратным и понятным.
- Валидация пользовательского ввода **обязательна**.
- Не используйте `document.write()`.
- Не используйте inline-события в HTML (`onclick="..."`). Все обработчики — через `addEventListener` в `script.js`.
- Вывод результатов — на страницу (через `innerHTML` / `textContent`), не через `alert()`.
- Для сравнений используйте `===` и `!==`.
- Данные о товарах храните в массиве объектов (не в отдельных переменных).
- Весь JavaScript — в **script.js**. Никакого JS в HTML.
- **Комментарии «ПОЧЕМУ?»:** Возле каждого вызова `map()`, `filter()`, `forEach()` ОБЯЗАТЕЛЬНО должен быть комментарий: `// Использую map, потому что...` или `// filter подходит здесь, так как...`
- **УСТНАЯ ЗАЩИТА:** Преподаватель укажет на любую строку кода. Если вы не сможете объяснить, что делает метод и почему выбран именно он — работа не принимается (0 баллов).
- **ТАБУ:** Запрещено использовать циклы `for/while` там, где задание требует `map/filter/forEach`. Нарушение = минус уровень оценки.

Разметка страницы

Сделайте одну страницу с 4 секциями. Минимальный набор элементов (можно расширять):

- Секция «Каталог»: кнопка «Показать все товары» + блок для вывода карточек.
- Секция «Фильтры»: поле поиска по названию + select с категориями + кнопка «Найти».
- Секция «Корзина»: кнопка «Показать корзину» + блок для вывода выбранных товаров + итоговая сумма.
- Секция «Сохранение»: кнопки «Сохранить в localStorage» и «Загрузить из localStorage».

Стартовые данные (скопируйте в script.js)

Используйте этот массив как основу. Можно добавить свои товары (минимум 8 объектов):

```
const products = [
  { id: 1, name: "Ноутбук ASUS",      category: "Электроника", price: 2500, inStock: true  },
  { id: 2, name: "Мышь Logitech",     category: "Электроника", price: 45,   inStock: true  },
  { id: 3, name: "Стол письменный",   category: "Мебель",      price: 320,  inStock: false },
  { id: 4, name: "Кресло офисное",    category: "Мебель",      price: 480,  inStock: true  },
  { id: 5, name: "Наушники Sony",     category: "Электроника", price: 180,  inStock: true  },
  { id: 6, name: "Книга «JS для всех»",category: "Книги",      price: 25,   inStock: true  },
  { id: 7, name: "Книга «Clean Code»",category: "Книги",       price: 30,   inStock: false },
  { id: 8, name: "Монитор LG 27"",   category: "Электроника", price: 750,  inStock: true  },
];

let cart = []; // корзина — тоже массив объектов
```

Задания («Каталог товаров»)

ЗАДАНИЯ ПО УРОВНЯМ СЛОЖНОСТИ

Выберите уровень в зависимости от вашей подготовки:

Уровень	Задания
🟢 БАЗА - для тех, кому тяжело (минимальные требования)	Задание 1 — вывод каталога через `forEach`
🟡 JUNIOR - стандартный уровень (рекомендуется всем)	Задание 2 — фильтр по категории через `filter`
	Задание 3 — поиск по названию + `map` для добавления поля `label`
	Задание 4 — корзина: добавление; `map` для суммы
🔴 PRO - для сильных студентов (дополнительные челленджи)	Задание 5 — `JSON` + `localStorage`: сохранить/загрузить корзину
	Задание 6 — статистика: средняя цена, самый дорогой, уникальные категории

🟢 УРОВЕНЬ «БАЗА» (Для тех, кому тяжело)

Минимальные требования для зачёта. Фокус на синтаксисе объектов и массивов.

Задание 1. Вывод каталога (forEach)

По нажатию кнопки «Показать все товары» выведите все товары на страницу.

Требования:

- Используйте `forEach` для перебора массива `products`.
- Для каждого товара создайте HTML-карточку с: названием, категорией, ценой (BYN), статусом наличия («В наличии» / «Нет в наличии»).
- Карточки вставьте в блок на странице через innerHTML.
- Товары, которых нет в наличии (`inStock`: `false`), выделите серым цветом или пометьте.

Подсказка — структура карточки:

```
// Пример одной карточки (шаблонная строка):
const card = `
  <div class="product-card">
    <h3>${product.name}</h3>
    <p>Категория: ${product.category}</p>
    <p>Цена: ${product.price} BYN</p>
    <p>${product.inStock ? "☑ В наличии" : "✕ Нет в наличии"}</p>
  </div>
`;
// Использую forEach, потому что: (ваш комментарий)
```

Задание 2. Фильтр по категории (filter)

Добавьте `<select>` с категориями: «Все», «Электроника», «Мебель», «Книги».

Требования:

- По нажатию кнопки «Найти» отфильтруйте товары по выбранной категории с помощью `filter()`.
- Если выбрано «Все» — показывайте все товары.
- Результат выводите в тот же блок каталога (перезаписывайте `innerHTML`).
- Если ничего не найдено — выведите сообщение «Товары не найдены».

Подсказка:

```
// filter возвращает НОВЫЙ массив — исходный products не меняется!
const filtered = products.filter(product => product.category === selectedCategory);
// Использую filter, потому что: (ваш комментарий)
```

🟡 УРОВЕНЬ «JUNIOR» (Стандарт)

Рекомендуемый уровень для всех. Фокус на реальных сценариях работы с данными.

Задание 3. Поиск по названию + сортировка (filter + map)

Добавьте поле ввода для поиска по названию товара.

Требования:

- Поиск должен работать без учёта регистра (`toLowerCase()`).
- Используйте `filter()` для отбора подходящих товаров.
- Используйте `map()` для преобразования найденных товаров: добавьте к каждому объекту поле `label` — строку вида «Ноутбук ASUS — 2500 BYN».

- Выведите результат на страницу.
- Комбинируйте фильтр по категории и поиск по названию одновременно.

Подсказка — цепочка методов:

```
const results = products
  .filter(p => p.name.toLowerCase().includes(searchText.toLowerCase()))
  .filter(p => selectedCategory === "Все" || p.category === selectedCategory)
  .map(p => ({
    ...p,
    label: `${p.name} — ${p.price} BYN`
  }));
// filter использую для отбора, map — для добавления поля label, потому что: (комментарий)
```

Задание 4. Корзина покупателя (map + forEach)

Реализуйте простую корзину.

Требования:

-
- На каждой карточке товара добавьте кнопку «В корзину» (только если `inStock: true`).
- По нажатию добавляйте товар в массив cart (если товара ещё нет в корзине).
- По нажатию «Показать корзину» выведите список товаров в корзине через `forEach`.
- Используйте `map()` для подсчёта итоговой суммы: получите массив цен и сложите их.
- Выведите итоговую сумму под списком корзины.
- Добавьте кнопку «Очистить корзину» (`cart = []`, перерисовать блок).

Подсказка — подсчёт суммы через `map`:

```
// Получаем массив цен и суммируем
const prices = cart.map(item => item.price);
const total = prices.reduce((sum, price) => sum + price, 0);
// Или в одну строку:
const total = cart.map(item => item.price).reduce((a, b) => a + b, 0);
// map использую здесь, потому что: (комментарий)
```

💬 ВОПРОСЫ ДЛЯ ИНТЕРВЬЮ (В README.md)

1. В чём разница между `map()` и `forEach()`? Когда использовать каждый?
2. Что возвращает `filter()`? Изменяет ли он исходный массив?
3. Что такое JSON? Чем JSON-строка отличается от JavaScript-объекта?
4. Что произойдёт, если передать в `JSON.parse()` невалидную строку?
5. Что такое деструктуризация объекта? Приведите пример из своего кода.
6. Что делает оператор `spread (...)` при копировании объекта?
7. Почему нельзя сравнивать объекты через ===? Что выведет: `{} === {}`?
8. Что такое метод `reduce()`? Как вы использовали его для подсчёта суммы?

Скриншоты (папка screenshots/)

- Каталог всех товаров (с карточками, включая недоступные).

- Результат фильтрации по категории.
- Результат поиска по названию.
- Корзина с товарами и итоговой суммой.
- Консоль с выводом `JSON.stringify` и `JSON.parse` (Задание 5).
- Блок статистики с рассчитанными данными (Задание 6).

Критерии оценки

- Проект запускается (Live Server), нет ошибок в Console при обычном сценарии.
- Каталог отображается корректно, недоступные товары помечены.
- Фильтр по категории и поиск по названию работают.
- Корзина: добавление, отображение, подсчёт суммы, очистка.
- `JSON`: сохранение в `localStorage` и загрузка работают корректно.
- В коде используются `forEach`, `map`, `filter` — по назначению, с комментариями.
- Репозиторий оформлен: структура, `README` с ответами, папка screenshots.

Самостоятельная работа (необязательно, но желательно)

🔴 УРОВЕНЬ «PRO» (Для сильных студентов)

Дополнительные баллы. Фокус на JSON и работе с реальными данными.

Задание 5. JSON и localStorage (JSON.stringify / JSON.parse)

Реализуйте сохранение и загрузку корзины через localStorage.

Требования:

- По нажатию «Сохранить в localStorage» — сохраните массив cart как JSON-строку.
- По нажатию «Загрузить из localStorage» — загрузите данные, распарсите JSON и отобразите корзину.
- Если localStorage пуст — выведите сообщение «Сохранённых данных нет».
- В консоли (console.log) покажите: исходный объект, JSON-строку и распарсенный объект — чтобы видеть разницу.

Подсказка:

```
// Сохранение
const cartJSON = JSON.stringify(cart);
localStorage.setItem("myCart", cartJSON);
console.log("Объект:", cart);
console.log("JSON-строка:", cartJSON);

// Загрузка
const saved = localStorage.getItem("myCart");
if (saved) {
  cart = JSON.parse(saved);
  console.log("Загружено:", cart);
} else {
  console.log("Нет сохранённых данных");
}
```

Задание 6. Статистика каталога (map + filter + вычисления)

Добавьте блок «Статистика» с автоматически рассчитанными данными.

Требования — выведите на страницу:

- Общее количество товаров.
- Количество товаров в наличии (filter по inStock).
- Среднюю цену всех товаров (map → цены → среднее, округлить до 2 знаков).
- Самый дорогой товар (название и цена).
- Список уникальных категорий (без повторений).

Подсказка — средняя цена:

```
const prices = products.map(p => p.price);
const avgPrice = (prices.reduce((a, b) => a + b, 0) / prices.length).toFixed(2);

// Самый дорогой:
const mostExpensive = products.reduce((max, p) => p.price > max.price ? p : max, prod-
ucts[0]);

// Уникальные категории:
const categories = [...new Set(products.map(p => p.category))];
```

Примеры для проверки

Действие	Ожидаемый результат
Нажать «Показать все товары»	8 карточек, 2 из них серые (нет в наличии)
Выбрать категорию «Электроника»	4 товара: Ноутбук, Мышь, Наушники, Монитор
Поиск: «книга»	2 товара: «Книга JS для всех», «Книга Clean Code»
Добавить Ноутбук и Мышь в корзину	Итого: 2545 BYN
Сохранить корзину → обновить страницу → загрузить	Корзина восстановлена из localStorage
Средняя цена всех товаров	541.25 BYN

Форма сдачи: Отправьте преподавателю ссылку на GitHub-репозиторий в Slack

Примечание: при необходимости вы можете упростить стили. Оценивается логика, корректность условий, обработка ошибок и аккуратность кода.